



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	MJ Programming from First Principles	
Topic Name	A means to extend the language (practice session), part 2	
Topic Code	-	
Unit/Module No. & Name	10	User-defined types
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4
2.0	Meaning And Definition	5-7
3.0	Nature Or Type	8-10
4.0	Examples And Case Studies	11-13
5.0	Keywords And Touch Vocabulary	14



Savitribai Phule Pune University
Centre For Distance and Online Education

OBJECTIVE

1. A data type defines what values a program can store and what operations can be done on those values.
2. Data types help catch mistakes early by preventing invalid operations.
3. User-defined types let programmers extend the language to match real-world needs, not just follow basic syntax.
4. This unit teaches you to think about design, not only to write code that runs.
5. You will learn to find the main ideas in a problem and represent them as types.
6. This way of thinking helps in advanced courses and real software work.
7. You will learn to read a problem and identify nouns that can become user-defined types.
8. You will be able to design simple types like Student, Book, or Track with useful fields.
9. You will understand the difference between built-in types and user-defined types in clear sentences.
10. You will learn to choose the right kind of user-defined type for the situation.
11. You will know to use an enumeration when you need a fixed list of named values.
12. You will know to use a record or struct when you need a simple bundle of fields.
13. You will know to use a class when you need both data and behavior together.
14. You will develop the ability to justify your design choices and match each type to the real problem.



1.0 INTRODUCTION

1.1 Why We Study Data Types

Every computer program works with some kind of data. A calculator uses numbers. A chat app uses messages, usernames, and time stamps. A music player uses songs, artists, and playlists. When you write a program, you must tell the computer what type of data you want to store and what you want to do with it. This job is done by a data type.

Most programming languages give you simple, basic types like integer, float, character, string, and Boolean. These are like the basic blocks used to build bigger things. They are useful but not enough to represent real-life objects. For example, a college student is not only a name — a student has an ID, courses, grades, contact details, and many more pieces of information.

User-defined types let you create your own custom blocks. You can group related data together and give that group a clear name. The computer then treats your group as a new type, just like an integer or string. In this way, you act like a small language designer inside your own project.

In beginner programming courses, students start with only primitive types. But as programs grow, they become hard to manage. Many variables start appearing together and must always stay connected. At this point, user-defined types become important. They make the code cleaner, more organized, and easier to understand.

1.1.1 Real World and Programs

There is a strong connection between the real world and the world inside a computer program. When we say modeling, we mean creating a simple version of a real thing so a computer can work with it. Just like a map is a small model of a real city, a user-defined type is a model of a real or imaginary thing in a program.

You cannot store a real person in a computer, but you can store the important details about that person. You choose which details to include and how to organize them. If this is done carefully, your type becomes simple enough for the machine, but useful enough for your problem. Good modeling means choosing what to include and also what to leave out.

2.0 MEANING AND DEFINITION

2.1 Meaning Of Data Type

In computer science, a data type is a rule that tells us what kind of value we have and what operations we can perform on that value. An integer type allows addition and subtraction. A string type allows operations like joining two strings or searching for a letter. The type helps the compiler or interpreter check that you are using values in a safe and meaningful way.

We can think of a type as a label plus a set of allowed actions. When we declare a variable with a type, we are saying, “This box will hold values of this shape, and I will use them in these ways.” If we try to use the variable in a wrong way, the type system can warn us. In this sense, the type system works like an assistant that looks over your code and points out likely mistakes.

2.1.1 Meaning Of User Defined Type

A user defined type is a data type that is created by the programmer, using language features such as structures, classes, records, or enumerations. While primitive types are supplied by the language designers, user defined types come from the needs of a specific problem. Whenever the simple types are not enough to express an idea clearly, the programmer can invent a new type that fits that idea.

We define a user defined type by giving it a name and describing its internal structure. The structure can be made from primitive types and from other user defined types. Once we have defined such a type, we can declare variables of that type, pass them to functions, return them from methods, and store them in collections. In this way, the new type becomes a full citizen of the language, almost as if the language had always had it.

2.2 Formal View Of User Defined Types

In a more formal view, a user defined type describes a set of possible values and a set of operations that make sense for those values. The values are built from other types using builders such as “record of,” “list of,” or “either this or that.” The operations may include reading and writing fields, comparing two values, or applying domain specific rules such as checking that a grade is in a valid range.

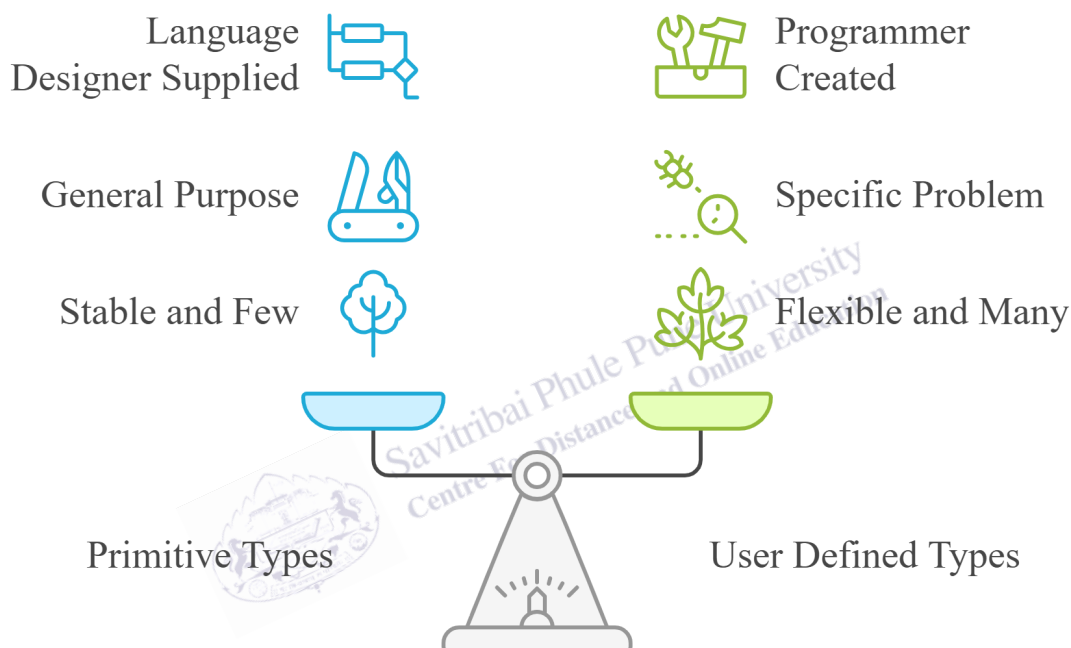
Formal views matter when we reason about program correctness. If we know that a certain type always has a nonempty list of items, we can prove that some functions never face an empty input. If we know that a field is an enumeration with only three states, we can check that our code handles all three states and does not forget a case. This logical view of types is part of what makes them powerful tools.

2.3 Built In And User Defined Types

Built in types are general tools that work in almost every program. They are few in number, stable, and highly tuned by the language implementers. User defined types are custom tools that match one family of problems well. They can be many, flexible, and ready to change as the design of the system changes.

Fig. 1

Balancing Primitive and User Defined Types



This diagram compares primitive and user-defined data types. Primitive types are language-provided, stable, and general-purpose, while user-defined types are created by programmers to solve specific problems, offering flexibility, customization, and the ability to model complex data and behavior.

In practice, we combine both. Primitive types handle low level details such as counts, indices, and simple flags. User defined types handle higher level ideas such as Customer, Order, Playlist, or Appointment. This layered use of types mirrors the way humans think, moving from concrete numbers up to more abstract roles and relationships.

2.4 Role Of The Type System

A type system is the set of rules that a language uses to connect types with values and expressions. Some type systems are static, meaning they check most rules before the program runs. Others are dynamic, meaning they check many rules while the program is running. Many modern languages mix both ideas so that they can catch many errors early and still remain flexible.

For user defined types, the type system helps catch mismatches. If a function expects a Track and you pass a Patient, the compiler can stop you. If a field is declared as an enumeration and you try to store a number outside the allowed set, the system can complain. In this way, the type system supports the programmer and gives extra safety, especially as the program grows large.



3.0 NATURE OR TYPE

3.1 Main Kinds Of User Defined Types

User defined types appear in many forms, depending on the language and the style of programming. The most common forms are records or structs, classes and objects, enumerations, and variants or unions. Each form has its own nature and is suited to certain modeling tasks. Learning when to use each kind is part of learning to extend the language wisely.

When you design a system, it helps to know which form to choose. A record is simple and close to the data. A class adds behavior and hiding of details. An enumeration gives a short list of states. A variant lets a value take one of several shapes. Together, they form a small toolbox for modeling many different problems in a tidy way.

3.1.1 Records Or Structs

A record, often called a struct in languages like C, is a simple user defined type that groups several named fields. Each field has its own type. All fields are stored together in memory, often in a fixed layout. Records are good for modeling simple objects that mostly store data and do not need complex behavior.

For example, a record for a student might contain an integer field for the student id, a string field for the name, and another string field for the major. Functions that work on students can take a record as a parameter, but the record itself does not carry methods. This keeps the data and behavior separate and easy to see.

3.1.2 Classes And Objects

A class is a more powerful user defined type that supports both data and methods in one unit. An object is a concrete instance of a class. Classes are central to object oriented programming languages. They allow ideas like encapsulation, which means hiding inner details, and inheritance, which means sharing behavior between related classes.

When we say that user defined types are a means to extend the language, classes are often what we imagine. A class can hide its internal fields and show only a clean interface. This gives other parts of the program a simple way to use the model without needing to know how it is built inside. In this way, a well designed class acts like a new word in the language that many parts of the program can use.

3.1.3 Enumerations

An enumeration, or enum, is a user defined type that lists a small set of named values. It is useful when a variable can only take one value from a limited list, such as days of the week, levels of

access, or states of a process. Enums make code safer and clearer because they avoid magic numbers and repeated string names that can be typed wrongly.

Fig. 2

Comparison of User-Defined Types

Characteristic	Data Storage	Behavior	Mutability	Use Case
Records/Structs	Groups named fields	Mostly store data	Can be mutable or immutable	Modeling simple objects
Classes/Objects	Data and methods in one	Supports data and methods	Can be mutable or immutable	Extending the language
Enumerations	Lists named values	Avoids magic numbers	Immutable	Representing limited options
Variants/Unions	Several different shapes	Models related data	Can be mutable or immutable	Representing multiple forms

This table compares user-defined data types by showing how records, classes, enumerations, and variants differ in data storage, behavior, mutability, and use cases. It helps students choose the right type to model simple objects, structured data, limited options, or multiple forms.

For example, instead of using 0, 1, and 2 to mean Stopped, Playing, and Paused, a music player program can define an enum called PlayerState with members STOPPED, PLAYING, and PAUSED. The compiler can then check that only these states are used in the code. Reading such code feels like reading short English phrases, which shows again how user defined types extend the language.

3.1.4 Variants And Unions

A variant or union type allows a value to take one of several different shapes. At any time, the value has one active form, and the program must know which form is in use. Modern languages often provide safe variant types, sometimes called tagged unions, which carry both the data and a tag that says which case is active at the moment.

Variants are helpful when modeling data that can appear in several related forms. For example, a program that reads media files could define a MediaItem variant that can represent an MP3 track,

a video clip, or a podcast episode. Each case can have its own fields, such as resolution for video or series title for podcasts, but all cases share the same overall type.

3.2 Mutable And Immutable Types

Another part of the nature of a type is whether its values can change after creation. A mutable type allows its fields to be updated. An immutable type does not allow such changes; instead, any update creates a new value. Immutable types are common in functional programming and can reduce bugs related to unexpected changes in shared data.

For user defined types, you should decide which fields, if any, may change. For example, a Patient type might allow the address to change over time, but it should not allow the id or date of birth to change. This decision can be enforced by the language or followed as a design rule in the team. Thinking about mutability is another way to shape how your types match the real world.



4.0 EXAMPLES AND CASE STUDIES

4.1 Simple Examples

To see how user defined types help us model real situations, it is useful to start with a few simple examples. These examples are written in plain language rather than in a specific coding syntax, so that the ideas remain clear and easy to follow. Each example shows how a type brings scattered pieces of data together into a single, named unit.

4.1.1 Example Of A Student Type

Suppose we want to build a small system to track students in a course. If we try to use only primitive types, we might have separate arrays for student ids, names, and grades. This is hard to manage and easy to break, because a mistake in indexing can mix up one student with another.

Instead, we define a user defined type named Student with fields such as id, name, email, and grade. Now a single Student value keeps related data together. When we pass a Student to a function that prints a report or updates a grade, we know that all needed data travels as a unit. The Student type becomes a new word in the language of the program.

4.1.2 Example Of A Book Type

In a library system, we can define a Book type with fields like title, author, publicationYear, and isAvailable. We might also attach methods such as issueToMember and returnToShelf if we use classes. Each Book instance then models one real book in the library. The type definition acts as a template from which many book objects can be created and stored.

4.2 Case Study Of A Music Library

Now we move to a larger case study. Imagine we are designing a simple MP3 music library application. The goal of the system is to store information about MP3 tracks, show them to the user, and play them on request. User defined types give us a way to model each part of this system in a direct and readable way, so that the code and the user's view of the library stay close to each other.

4.2.1 Mp3 Track Type

We can define a type called Track with fields such as title, artist, album, durationInSeconds, and filePath. Each Track stands for one MP3 file stored on disk. The filePath points to the real file. The other fields store the metadata that a user cares about when they search, sort, or play songs. Because Track is a user defined type, it can also have methods such as play, pause, and displayMetadata in an object oriented design.

4.2.2 Playlist Type

A music library also needs playlists. A Playlist type can have a name and an ordered list of Track values. Methods on Playlist can add a track, remove a track, or move a track up or down. The Player part of the program can then use these methods to move through the playlist in order. Together, the Track type and the Playlist type extend the language of the program so that it can talk about music in a natural way.

4.3 Case Study Of A Hospital System

Our second case study is a small hospital information system. Here, the key actors are patients, doctors, and appointments. User defined types help us keep each of these ideas clear and separate while also linking them in the right way. Again, we see that the types extend the language of the program to match the language of the hospital staff.

4.3.1 Patient, Doctor, And Appointment Types

We can define a Patient type that stores id, name, dateOfBirth, bloodGroup, and address. A Doctor type may have id, name, specialty, and a list of assigned patients. An Appointment type can connect a patient, a doctor, a date, and a status field that uses an enumeration such as SCHEDULED, COMPLETED, or CANCELLED.

By tying these types together, we get a rich model of the system. When a new appointment is created, the program creates an Appointment value that links to one Patient and one Doctor. When we print a schedule, we loop over Appointment values rather than over separate lists of names and times. The program can now speak in terms of Patients, Doctors, and Appointments, which shows how user defined types extend its language.

4.4 Common Mistakes In Modeling

Beginners often make a few common mistakes when designing user defined types. One mistake is to use only primitive types, even when a new type would make the code clearer. Another mistake is the opposite: creating too many tiny types that do not add meaning, such as wrapping an integer id in a separate type that is never given more structure or behavior.

A further mistake is to mix unrelated ideas in one type. For example, a Student type that also stores details about the last exam room and the cafeteria balance may be carrying too much. In such cases, it is better to create smaller types, such as ExamRecord and PaymentAccount, and link them to the Student type. Good modeling tries to give each type one main purpose.

4.5 Reflection Activities

To deepen your understanding, you can try some short reflection activities. Take a real system you know well, such as your college portal, a shopping app, or a music streaming service. On a sheet of paper, list the main nouns that appear on the screen. These may include words like user, order, product, playlist, or course. For each noun, try to imagine a user defined type that could model it inside the code.

These ideas prepare you for later courses in software design and give you a steady way to think about programs of any size and style today.

1. User defined types extend a language so that it can model real world ideas.
2. Primitive types are basic building blocks, while user defined types build larger models.
3. Records, classes, enums, and variants are common forms of user defined types.
4. Type systems help catch mismatches and protect programs from many errors.
5. Good modeling keeps each type focused on one main purpose and avoids mixing unrelated ideas.



5.0 Keyword Touch Vocabulary Meaning

Keyword	Simple Meaning In This Unit
Data type	Rule about what kind of value we have and how we use it
Primitive type	Basic built in type such as integer, float, or Boolean
User defined type	New type created by the programmer for a specific problem
Record / struct	Group of named fields stored together as one value
Class	Type that holds data and related methods
Object	Actual instance of a class in a running program
Enumeration (enum)	Type with a small fixed set of named values
Variant / union	Type whose value can take one of several shapes
Type system	Set of rules that connect types with values and programs
Playlist	Ordered list of track values in a music library
Appointment	Link between patient, doctor, date, and status